will be forced to login again every time they open a new tab and logging out of one tab will not log them out of the others. For this reason, it is more convenient to store tokens in `localStorage` instead.

Each object implements the same `Storage` interface that defines `setItem(key, value)`, `getItem(key)`, and `removeItem(key)` methods to manipulate key-value pairs in that storage. Each storage object is implicitly scoped to the origin of the script that calls the API, so a script from example.com will see a completely different copy of the storage to a script from example.org.

> **TIP** If you want scripts from two sibling sub-domains to share storage, you can set the `document.domain` field to a common parent domain in both scripts. Both scripts must explicitly set the `document.domain`, otherwise it will be ignored. For example, if a script from a.example.com and a script from b.example.com both set `document.domain` to example.com, then they will share Web Storage. This is allowed only for a valid parent domain of the script origin, and you cannot set it to a top-level domain like com or org. Setting the `document.domain` field also instructs the browser to ignore the port when comparing origins.

To update the login UI to set the token in local storage rather than a cookie, open login.js in your editor and locate the line that currently sets the cookie:

```
    document.cookie = 'token=' + json.token +
        ';Secure;SameSite=strict';
```